

that the broken valve is repaired. AquaLush begins using it within one minute of being informed that it is repaired.

Configure at Startup—The AquaLush program loads and executes. AquaLush is able to communicate with all valves and sensors.

Simulate AquaLush—A Web user brings up the AquaLush simulation. The user manipulates the simulation. The simulation works just the way the fielded AquaLush product does.

Hardware Adaptability Profile

Add a Valve Type—A developer modifies the AquaLush source code and design documents so that AquaLush supports a new type of valve. The developer completes the process in two days.

Replace the Keypad—A new brand of keypad is used in the control panel. A developer is able to modify the source code to use the new keypad within two days.

Modifiability Profile

Add Timer-Based Irrigation—A new timer-based mode is added to AquaLush. When in timer-based mode, AquaLush starts an automatic irrigation cycle at the preset irrigation time and irrigates each zone for a set period of time or until the zone's water allocation is exceeded. A developer is able to modify the design and source code to add this mode in two weeks.

Modify Irrigation Process—The automatic irrigation process is changed to allow users to specify concurrent irrigation of two or more zones. A developer is able to modify the design and source code to make this change in three weeks.

Reusability Profile

In an Agricultural Product—VIS develops a new product especially for an agricultural market. Developers are able to reuse at least 60% of the design and code from AquaLush.

In a Consumer Product—VIS develops a new product especially for the home. Developers are able to reuse at least 90% of the design and code from AquaLush.

Reliability Profile

Power Fails—Power is applied to AquaLush after a power failure. AquaLush returns to its state at the time of the power failure, except that AquaLush is not able to set its clock and, if an irrigation cycle was underway when the power failed, it is abandoned. AquaLush closes all irrigation valves.

A Sensor Fails—A sensor fails during automatic irrigation. AquaLush closes all valves in that zone within one minute and goes to the next zone. AquaLush marks the sensor as broken and does not attempt to use it again until told that it is repaired by a user.

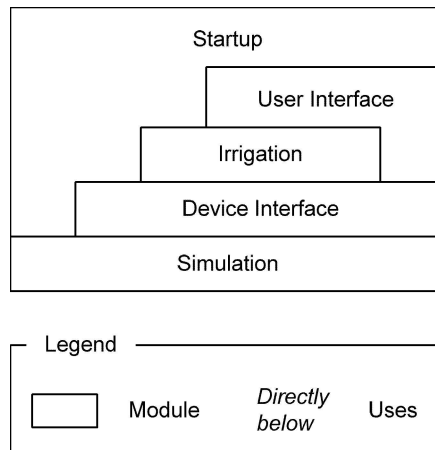
B.10 AquaLush Software Architecture Document

1. Product Overview

Please see section B.2 *AquaLush Project Mission Statement* for an overview of the product.

2. Architectural Models

AquaLush is a Relaxed Layered system, as shown in the layer diagram in Figure B-10-1.

**Figure B-10-1 AquaLush Layers Overview**

Modules use only layers below them that they touch directly. A fielded system does not include the bottom **Simulation** layer, which is needed only as a stand-in for the hardware that would be present in a fielded system.

2.1 Layer Responsibilities

Layer	Responsibilities
Startup	Create and connect all runtime components based on a configuration specification and whether the program is a simulation or a fielded product. Restore the state of the system from persistent store.
User Interface	Implement a device-independent AquaLush text-based user interface.
Irrigation	Control both manual and automatic irrigation.
Device Interface	Implement virtual devices providing interfaces to all external entities, including hardware devices, a clock, and persistent storage.
Simulation	Implement components simulating hardware devices, simulate an irrigation site, and provide views and controls for manipulating the simulation.

Table B-10-2 AquaLush Layer Responsibilities

2.2 Startup Layer Decomposition

The **Startup** layer contains the main program class, which may be either an application or an applet, assuming the program is written in Java. The main program uses a **Configurer**, which sets up the program based on the available hardware (real or simulated) and whether it is a simulation or a fielded program, and restores the previous program state. The **Configurer** reads a configuration specification to determine what sort of hardware (or simulated hardware) the program must use. It reads a persistent store to reset the program state. Figure B-10-3 shows the decomposition of the **Startup** layer.

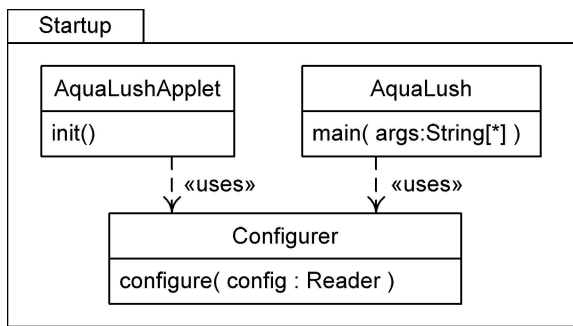


Figure B-10-3 Startup Layer Structure

2.2.1 Startup Layer Module Responsibilities

Module	Responsibilities
AquaLushApplet	Ask the Simulation layer to create all GUI components and simulated entities, and create a Configurer that uses these simulation objects to configure the program. Obtain the configuration specification. Call the Configurer to configure the program.
AquaLush	Create a Configurer that uses real hardware. Obtain the configuration specifications. Call the Configurer to configure the program.
Configurer	Create and connect runtime components based on configuration specifications and whether the program is a simulation or a fielded product. Restore the state of the system from persistent store.

Table B-10-4 Startup Layer Module Responsibilities

2.2.2 Startup Layer Interface Specifications

Services Provided		
Applet initialization	<i>Syntax:</i>	init()
	<i>Pre:</i>	None.
	<i>Post:</i>	The applet simulating AquaLush is initialized.
Program execution	<i>Syntax:</i>	main(args : String[])
	<i>Pre:</i>	None.
	<i>Post:</i>	The AquaLush program is set up according to its local configuration and its state is reset to what it was when it was last executing.

Table B-10-5 Startup Layer Interface Specifications

Services Required

Simulation.create()—Create the AquaLush simulation, including GUI components simulating the control panel display, keypad, and screen buttons; the simulated irrigation site with valves and sensors; and simulated time and persistent store objects.

DeviceFactory.createDeviceX()—Create various virtual devices already connected to real or simulated hardware.

DeviceX.setListener()—Add a listener to a keypad or screen button virtual device.

Irrigator.create()—Create irrigation control objects.

`Irrigator.addX()`—Configure the `Irrigator` with zones, sensors, and valves, and configure the virtual devices controlling valve and sensor hardware.

`Irrigator.restoreState()`—Restore the state of the program from persistent store.

`UI.setDisplay()`—Create the user interface.

`UI.setDisplay()`—Configure the user interface with the virtual display.

`Reader.read()`—Obtain an input character from the configuration specification.

2.2.3 Startup Layer Design Rationale

The configuration task could have been placed in the applet and application classes, but then there would have been two versions of very similar code. Separating this task into a separate component (the `Configurer`) that can be reused makes it easier to change and correct as well as faster to implement, though it takes more work to design originally. Note: This component will make use of the Abstract Factory pattern for virtual devices, which we discuss in the next section.

2.3 Simulation Layer Decomposition

The `Simulation` layer contains components that simulate the entire external environment during a simulation. Five of these are externally visible: a time simulator, a persistent storage simulator, a display device simulator, and sensor and valve simulators. In addition this layer has a façade class that hides other components (such as a simulated keypad, simulated screen buttons, the simulated site, and so forth). These components are pictured in Figure B-10-6.

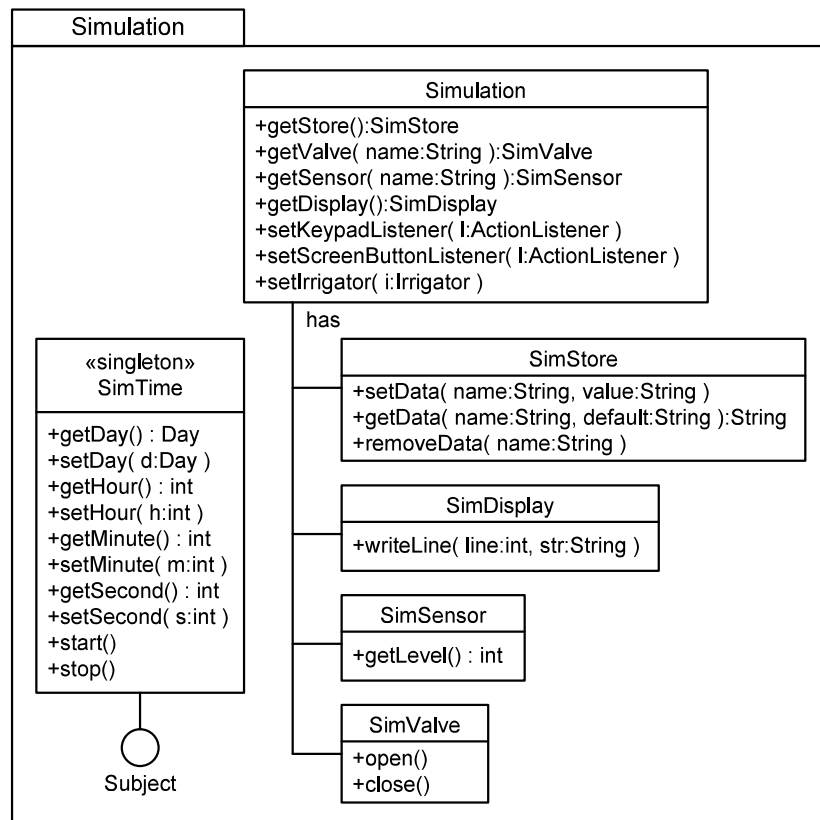


Figure B-10-6 Simulation Layer Structure

2.3.1 Simulation Layer Module Responsibilities

Module	Responsibilities
Simulation	Simulate hardware devices and the real world, and display the simulation using GUI components.
SimStore	Simulate a persistent store. A real file will be used if possible; if not, no real persistence will be simulated.
SimDisplay	Simulate a 16-line-by-40-character monochrome textual display as a GUI component.
SimSensor	Simulate a moisture sensor.
SimValve	Simulate an irrigation valve.
SimTime	Simulate the passage of time. Notify other components of the passage of simulated time.

Table B-10-7 Simulation Layer Module Responsibilities**2.3.2 Simulation Layer Interface Specifications**

Services Provided

All operations with preconditions on parameters throw `IllegalArgumentException`s if the preconditions are violated.

Create the simulation object	<i>Syntax:</i>	<code>create()</code>
	<i>Pre:</i>	None.
	<i>Post:</i>	All simulated entities are created.
Provide simulated persistent store	<i>Syntax:</i>	<code>getStore() : SimStore</code>
	<i>Pre:</i>	None.
	<i>Post:</i>	The simulated storage object is returned.
Provide a simulated valve	<i>Syntax:</i>	<code>getValve(name : String) : SimValve</code>
	<i>Pre:</i>	name is not null.
	<i>Post:</i>	The simulated valve whose identifier is name is returned, or null is returned if no such valve exists.
Provide a simulated sensor	<i>Syntax:</i>	<code>getSensor(name : String) : SimSensor</code>
	<i>Pre:</i>	name is not null.
	<i>Post:</i>	The simulated sensor whose identifier is name is returned, or null is returned if no such sensor exists.
Provide the simulated display	<i>Syntax:</i>	<code>getDisplay() : SimDisplay</code>
	<i>Pre:</i>	None.
	<i>Post:</i>	The simulated display object is returned.
Register a keypad listener	<i>Syntax:</i>	<code>setKeypadListener(l : KeypadListener)</code>
	<i>Pre:</i>	None.
	<i>Post:</i>	KeypadListener l will start to receive notifications of simulated keypad key presses.
Register a screen button listener	<i>Syntax:</i>	<code>setScreenButtonListener(l : ScreenButtonListener)</code>
	<i>Pre:</i>	None.
	<i>Post:</i>	ScreenButtonListener l will start to receive notifications of simulated screen button presses.

Record a value in simulated persistent store	<i>Syntax:</i>	setData(name : String, value : String)
	<i>Pre:</i>	name and value are not null.
	<i>Post:</i>	The name-value pair is recorded in persistent store. Throws DeviceFailureException if persistent store cannot be read.
Fetch a value from simulated persistent store	<i>Syntax:</i>	getData(name : String, default : String) : String
	<i>Pre:</i>	name is not null.
	<i>Post:</i>	Returns the value associated with the name in persistent store. If no value is associated with name and default is not null, then associates the default with the name and returns it. If no value is associated with the name and the default is null, then returns null. Throws DeviceFailureException if persistent store cannot be read.
Remove data from simulated persistent store	<i>Syntax:</i>	removeData(name : String)
	<i>Pre:</i>	name is not null.
	<i>Post:</i>	The name and associated value are removed from persistent store. Throws DeviceFailureException if persistent store cannot be read.
Write to the simulated display	<i>Syntax:</i>	write(line : int, str : String)
	<i>Pre:</i>	0 <= line < 16 and str is not null.
	<i>Post:</i>	String str is written to line number line of the simulated display, starting at position 0.
Read a simulated sensor	<i>Syntax:</i>	getLevel() : int throws DeviceFailureException
	<i>Pre:</i>	None.
	<i>Post:</i>	The simulated moisture level for the sensor is returned. Throws DeviceFailureException if the sensor has a simulated failure.
Open a simulated valve	<i>Syntax:</i>	open() throws DeviceFailureException
	<i>Pre:</i>	None.
	<i>Post:</i>	Opens a simulated valve. Throws DeviceFailureException if the valve has a simulated failure.
Close a simulated valve	<i>Syntax:</i>	close() throws DeviceFailureException
	<i>Pre:</i>	None.
	<i>Post:</i>	Closes a simulated valve. Throws DeviceFailureException if the valve has a simulated failure.
Fetch the instance of SimTime	<i>Syntax:</i>	SimTime.instance() : SimTime
	<i>Pre:</i>	None.
	<i>Post:</i>	A single instance of the SimTime object will always be returned. The SimTime object always starts off with the current real time, but is stopped.
Get the simulated time	<i>Syntax:</i>	getDay() : Day getHour() : int getMinute() : int getSecond() : int
	<i>Pre:</i>	None.
	<i>Post:</i>	Returns the requested component of the simulated time.

Set the simulated time	<i>Syntax:</i>	setDay(d : Day) setHour(h : int) setMinute(h : int) setSecond(s : int)
	<i>Pre:</i>	0 <= h < 24 and 0 <= m, s < 60.
	<i>Post:</i>	The simulated time is adjusted as indicated.
Start and stop the simulated time	<i>Syntax:</i>	start() stop()
	<i>Pre:</i>	None.
	<i>Post:</i>	The simulated time is paused or resumed. Starting simulated time when it is already going or stopping it when it is already stopped has no effect.
Time notification	<i>Syntax:</i>	addObserver(o : Observer)
	<i>Pre:</i>	None.
	<i>Post:</i>	Observer o will be notified when simulated time is not stopped and a simulated second has passed.

Table B-10-8 Simulation Layer Interface Specifications**Services Required**

The **Simulation** layer requires the services of a GUI toolkit (such as Java Swing). It requires a **Day** enumeration type. It also requires a mechanism for regular notification of the passage of time occurring at least once per second and ideally much faster so that simulated time can be sped up.

Usage Guide

The **Simulation** layer is needed only when AquaLush is being simulated. Create it first because it is used in every other layer and in constructing the objects in almost every other layer.

2.3.3 Simulation Layer Design Rationale

Many alternatives considered for the **Simulation** layer lacked various components or had functionality present in other layers. For example, the persistent store was not initially in this layer until it was realized that an applet may not be allowed to access a host operating system because of security concerns. Eventually it was realized that this layer needs to simulate *everything* in the environment of the software. As another example, in one alternative the simulation GUI construction process was left partly up to the applet, so that the applet requested a simulated control panel and a simulated environment from the **Simulation** layer, then combined them to form the applet. The present alternative is more cohesive.

The **SimTime** class is a singleton class. This was deemed necessary because having more than one simulated time object would certainly cause the program to fail. Once **SimTime** is made a singleton there is no need for an operation to retrieve it from the **Simulation** layer façade, the role played by the **Simulation** class.

SimTime is also a subject in the Observer pattern. If it were an invoker in the Command pattern, then it could have only a single reactor. However, several components in the simulation as well as the **ClockDevice** in the **Device Interface** layer need to be notified of the passage of simulated time.

2.4 Device Interface Layer Decomposition

The **Device Interface** layer provides virtual devices to hide the real or simulated external entities manipulated by the program. Writing the program to interact only with virtual devices makes AquaLush highly configurable. The virtual devices provided in this layer are pictured in Figure B-10-9.

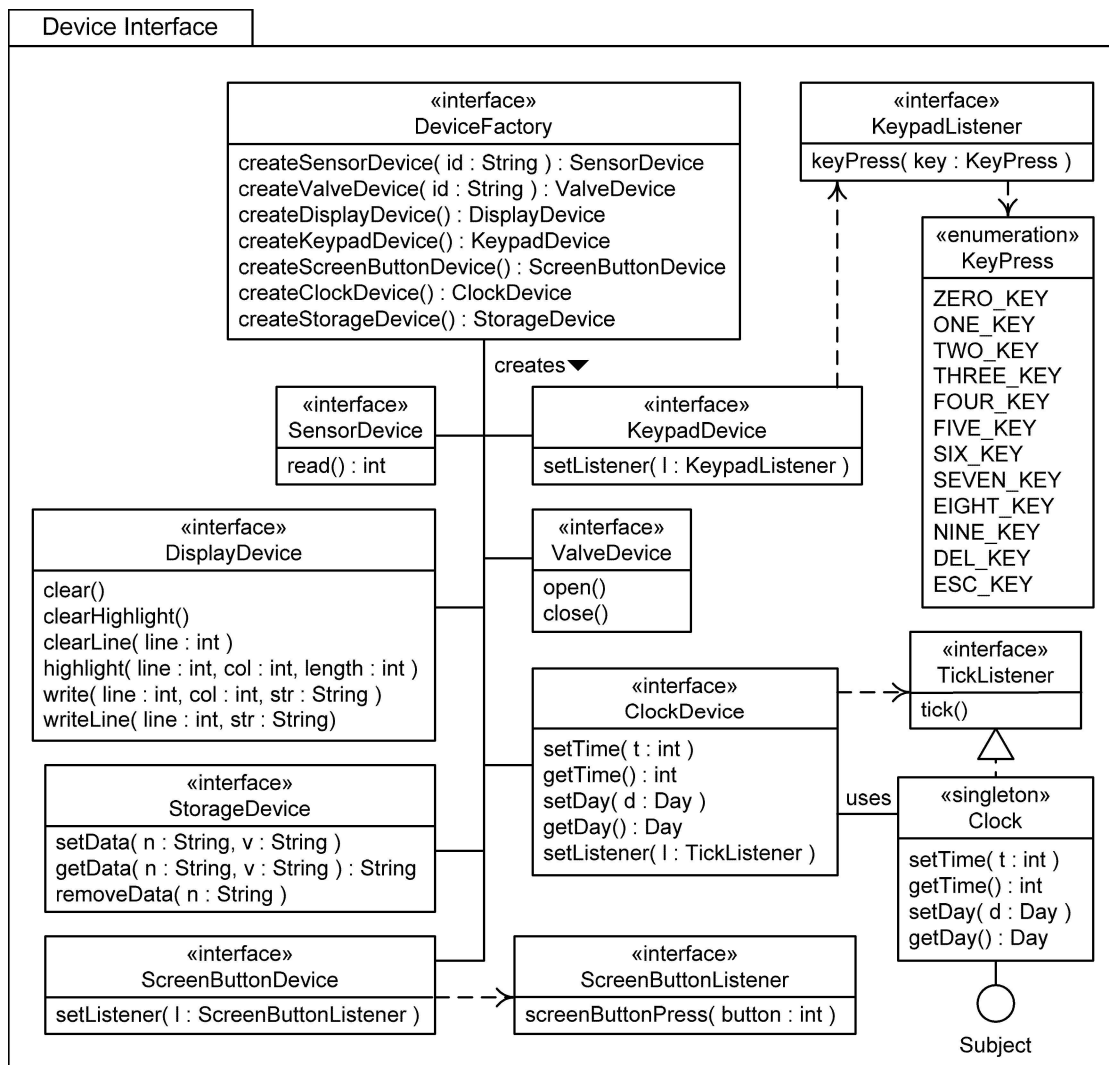


Figure B-10-9 Device Interface Layer Structure

Note that these entities are all interface types except for the **Clock** and the **KeyPress** enumeration. This layer also includes private realizations of these interfaces that actually do the work, but these are hidden at the architectural level of detail.

2.4.1 Device Interface Layer Module Responsibilities

Module	Responsibilities
ClockDevice Interface	Provide virtual clock hardware that keeps track of the day of the week and the time of the day, accurate to the minute.
TickListener Interface	Guarantee that a module listening to the ClockDevice implements a tick() operation, which is needed for the Command pattern.
StoreDevice Interface	Provide virtual persistent storage of name-value pairs.
DisplayDevice Interface	Provide a virtual 16-line-by-40-character monochrome textual display device.
SensorDevice Interface	Provide virtual moisture sensor hardware.
ValveDevice Interface	Provide virtual irrigation valve hardware.
KeypadDevice Interface	Provide a virtual 12-key keypad hardware device.
KeypadListener Interface	Guarantee that a module registered as a Keypad listener implements the keyPress() operation, which is needed for the Command pattern.
ScreenButtonDevice Interface	Provide a virtual hardware device with eight push buttons.
ScreenButtonListener Interface	Guarantee that a module registered as a ScreenButton listener implements the screenButtonPress() operation, which is needed for the Command pattern.
Clock	Provide the day of the week and the time of day, accurate to one minute. Notify observers of the passage of time every minute. The Clock should be used by the rest of the system; it relies on the ClockDevice.
DeviceFactory Interface	Provide abstract factory methods for creating all virtual devices. This interface is needed as part of the Factory Method pattern.

Table B-10-10 Device Interface Layer Module Responsibilities**2.4.2 Device Interface Layer Interface Specifications**

Services Provided

All operations with preconditions on parameters throw `IllegalArgumentException`s if the precondition is violated.

Set the time of the day (Clock and ClockDevice)	<i>Syntax:</i>	setTime(milTime : int)
	<i>Pre:</i>	milTime is a legitimate military time specification.
	<i>Post:</i>	The clock device is reset to milTime.
Get the time of the day (Clock and ClockDevice)	<i>Syntax:</i>	getTime() : int
	<i>Pre:</i>	None.
	<i>Post:</i>	The current time is returned, accurate to the minute, in military time format.
Set the day of the week (Clock and ClockDevice)	<i>Syntax:</i>	setDay(d : Day)
	<i>Pre:</i>	None.
	<i>Post:</i>	The clock device is reset to day d.

Get the day of the week (Clock and ClockDevice)	<i>Syntax:</i>	getDay() : Day
	<i>Pre:</i>	None.
	<i>Post:</i>	The current day of the week is returned.
Set the ClockDevice listener	<i>Syntax:</i>	setListener(l : TickListener)
	<i>Pre:</i>	None.
	<i>Post:</i>	TickListener l will start to receive notifications of the passage of time every minute. Any previous TickListener is replaced.
Register a Clock observer	<i>Syntax:</i>	addObserver(o : Observer)
	<i>Pre:</i>	o is not null.
	<i>Post:</i>	Observer o will start to receive notifications every minute. Throws an exception if the precondition is violated.
Register a ScreenButton listener	<i>Syntax:</i>	setScreenButtonListener(l : ScreenButtonListener)
	<i>Pre:</i>	None.
	<i>Post:</i>	ScreenButtonListener l will start to receive notifications of screen button presses.
Register a Keypad listener	<i>Syntax:</i>	setKeypadListener(l : KeypadListener)
	<i>Pre:</i>	None.
	<i>Post:</i>	KeypadListener l will start to receive notifications of keypad button presses.
Record a value in persistent store	<i>Syntax:</i>	setData(n : String, v : String)
	<i>Pre:</i>	n and v are not null.
	<i>Post:</i>	The name-value pair is recorded in persistent store.
Fetch a value from persistent store	<i>Syntax:</i>	getData(n : String, v : String) : String
	<i>Pre:</i>	n is not null.
	<i>Post:</i>	Return the value associated with n in persistent store. If no value is associated with n and v is not null, then associate v with n and return v. If no value is associated with n and v is null, return null.
Remove data from persistent store	<i>Syntax:</i>	removeData(n : String)
	<i>Pre:</i>	n is not null.
	<i>Post:</i>	Remove the association between n and a value (if any) from persistent store.
Clear the display	<i>Syntax:</i>	clear()
	<i>Pre:</i>	None.
	<i>Post:</i>	The display is blanked.
Clear all display highlighting	<i>Syntax:</i>	clearHighlight()
	<i>Pre:</i>	None.
	<i>Post:</i>	All highlighted lines are returned to regular display; the text is unaltered.
Clear a line of the display	<i>Syntax:</i>	clearLine(line : int)
	<i>Pre:</i>	0 <= line < 16.
	<i>Post:</i>	The text (and highlighting) are removed from the designated line.

Highlight a display line	<i>Syntax:</i>	highlight(line : int, col : int, length : int)
	<i>Pre:</i>	0 <= line < 16 and 0 <= col < 40 and 0 <= length.
	<i>Post:</i>	The portion of line starting at the column and extending for the length of the characters (or until the end of the line) is highlighted.
Write a string to the display	<i>Syntax:</i>	write(line : int, col : int, str : String)
	<i>Pre:</i>	0 <= line < 16 and 0 <= col <= 40 and str is not null.
	<i>Post:</i>	String str is written to line number line and column col of the simulated display. If str is too long to fit on the line it is truncated to fit. The line is not highlighted.
Write a line to the display	<i>Syntax:</i>	writeLine(line : int, str : String)
	<i>Pre:</i>	0 <= line < 16 and str is not null.
	<i>Post:</i>	String str is written to line number line of the simulated display, starting at position 0. If str is more than 40 characters it is truncated to 40 characters. If it is less than 40 characters, then it is padded with blanks until it is 40 characters. The line is not highlighted.
Read a sensor	<i>Syntax:</i>	read() : int throws DeviceFailureException
	<i>Pre:</i>	None.
	<i>Post:</i>	Returns the moisture level. Throws DeviceFailureException if the sensor fails.
Open a valve	<i>Syntax:</i>	open() throws DeviceFailureException
	<i>Pre:</i>	None.
	<i>Post:</i>	Opens a valve. Throws DeviceFailureException if the valve fails.
Close a valve	<i>Syntax:</i>	close() throws DeviceFailureException
	<i>Pre:</i>	None.
	<i>Post:</i>	Closes a valve. Throws DeviceFailureException if the valve fails.
Create virtual devices	<i>Syntax:</i>	createSensorDevice(id : String) : SensorDevice createValveDevice(id : String, rate) : ValveDevice createDisplayDevice() : DisplayDevice createKeypadDevice() : KeypadDevice createScreenButtonDevice() : ScreenButtonDevice createClockDevice() : ClockDevice createStorageDevice() : StorageDevice
	<i>Pre:</i>	id is not null.
	<i>Post:</i>	The desired virtual device is created. The virtual device is fully configured and connected to real or simulated hardware.

Table B-10-11 Device Interface Layer Interface Specifications**Services Required**

The **Device Interface** layer uses actual or simulated hardware. The details of these services vary: That is the point of having a device interface layer. This layer also requires a **Day** enumeration type and a **DeviceFailureException** class. The **ClockDevice** requires some sort of real or simulated device that notifies it when one minute has passed.

Usage Guide

The **Clock**, not the **ClockDevice**, is the main time and time notification service provider. The **Clock** is a singleton, so it is globally visible. It is also an observable, so it can be accessed from any layer and arbitrarily many observers can register with it for time notifications.

The extra parameter in `getData()` is there to make it easy to initialize the persistent store when the program is run for the first time. Default persistent values are supplied, and they will be written to the persistent store even if the store did not previously exist.

All sensors, valves, and zones are identified across layers using `String` identifiers. Hence these must match up and be hooked into whatever mechanism is used to associate virtual devices with real or simulated hardware.

2.4.3 Device Interface Layer Design Rationale

Other versions of this layer did not include the `DeviceFactory`. The `DeviceFactory` is part of the Abstract Factory pattern, which allows the details of virtual device configuration to be hidden inside concrete factories and makes it very easy to change the configuration. Design alternatives that include the Abstract Factory pattern are clearly superior.

A question that was revisited several times was how the clock should be handled. Does there need to be a `Clock` distinct from a `ClockDevice`? On the one hand, the `ClockDevice` is supposed to provide a virtual device as an interface to real or simulated hardware, while a `Clock` provides time and notification services to clients. These are distinct responsibilities, so it appears that there ought to be two entities. On the other hand, in practice the services provided by a `Clock` are almost identical to those of a `ClockDevice`, and it is not good to multiply entities beyond necessity. In the end the alternative with a distinct `Clock` and `ClockDevice` was chosen to maintain uniformity with the rest of the modules in the `Device Interface` layer.

Having resolved the issue of whether the `Clock` is distinct from the `ClockDevice`, the question of where the `Clock` should reside arises. On the one hand, the `Clock` is responsible for providing time and notification services rather than a uniform interface to hardware, suggesting that it should not be in the `Device Interface` layer. On the other hand, a `Clock` is a kind of device and it does not fit particularly well in any other layer, so perhaps the best home for it is the `Device Interface` layer. The latter argument seems slightly more persuasive, so the `Clock` is in the `Device Interface` layer. It could be moved easily, however.

2.5 Irrigation Layer Decomposition

The `Irrigation` layer is the core of AquaLush in the sense that it realizes the program's main function of controlling irrigation. Its architectural structure is quite simple: It contains a façade module that provides operations for configuration, setting irrigation parameters, and controlling manual irrigation. These operations depend on an enumeration type and several classes that merely package data about the state of the program for return from query functions. The class diagram in Figure B-10-12 illustrates this structure.

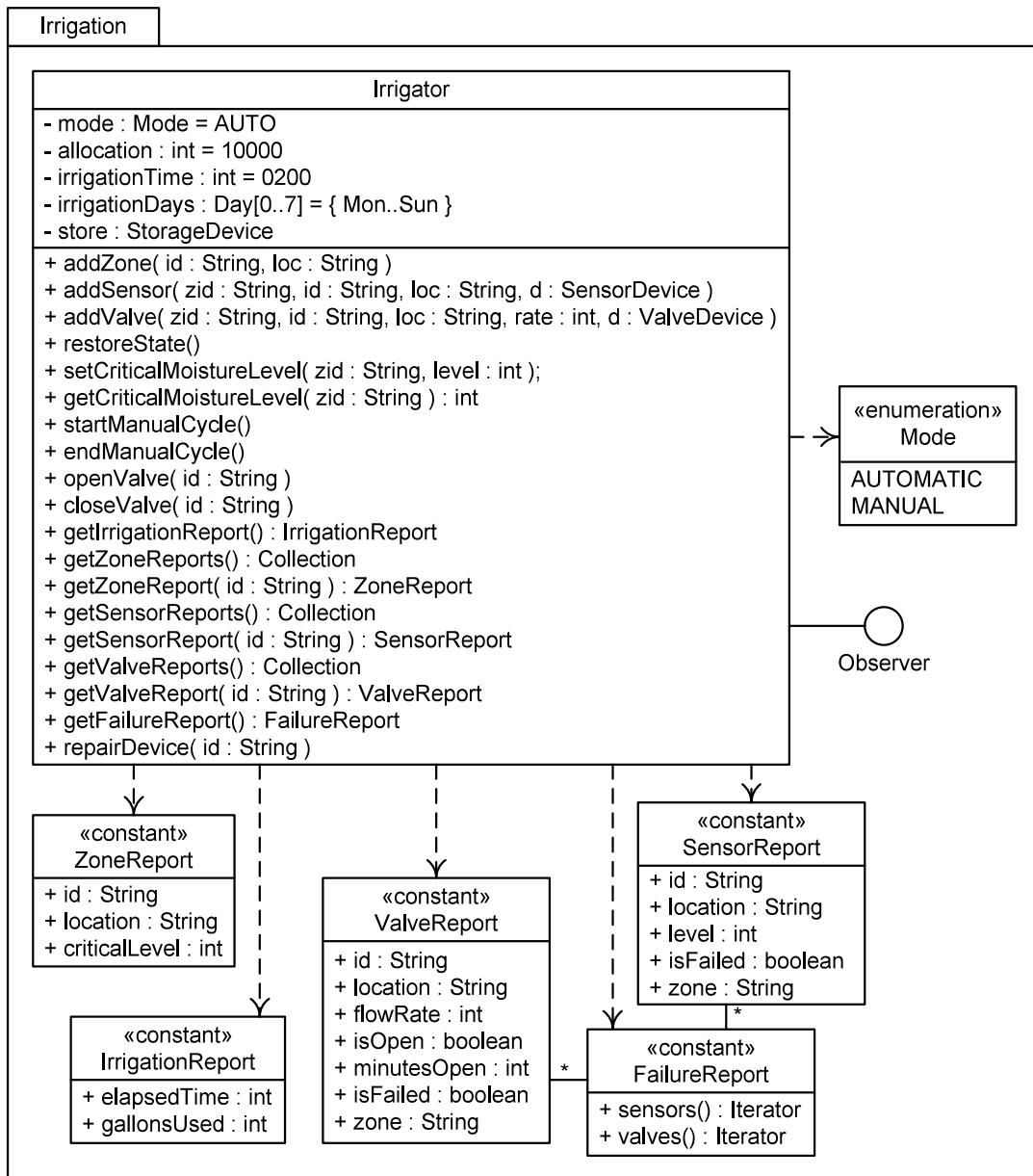


Figure B-10-12 Irrigation Layer Structure

The classes with the «constant» stereotype are immutable data containers that simply hold values returned by query operations.

2.5.1 Irrigation Layer Module Responsibilities

Module	Responsibilities
Irrigator	Hold irrigation parameters, oversee irrigation cycles, and provide a façade to the rest of the irrigation-layer facilities.
Mode	Enumeration values for program modes.
ZoneReport	Record providing data about irrigation zones.
ValveReport	Record providing data about the state of a valve.
SensorReport	Record providing data about the state of a sensor.
IrrigationReport	Record providing data about the state of an irrigation cycle.
FailureReport	Record containing ValveReport and SensorReport instances for failed valves and sensors.

Table B-10-13 Irrigation Layer Module Responsibilities

2.5.2 Irrigation Layer Interface Specifications

Services Provided

All operations with preconditions on parameters throw `IllegalArgumentException`s if the preconditions are violated.

Create irrigation objects	<i>Syntax:</i>	<code>create(s : StorageDevice)</code>
	<i>Pre:</i>	<code>s</code> is not null.
	<i>Post:</i>	Irrigation-layer objects are created and initialized.
Set and get irrigation parameters	<i>Syntax:</i>	<code>setMode(m : Mode)</code> <code>getMode() : Mode</code> <code>setAllocation(a : int)</code> <code>getAllocation() : int</code> <code>setIrrigationTime(t : int)</code> <code>getIrrigationTime() : t</code> <code>setIrrigationDays(s : Set)</code> <code>getIrrigationDays() : Set</code>
	<i>Pre:</i>	$0 < a$ and <code>t</code> is a valid military time specification and <code>s</code> is a set of <code>Day</code> .
	<i>Post:</i>	The irrigation parameters are set or fetched. Changes are recorded in the persistent store.
Add a zone	<i>Syntax:</i>	<code>addZone(id : String, loc : String)</code>
	<i>Pre:</i>	<code>id</code> and <code>loc</code> are not null; <code>id</code> is unique.
	<i>Post:</i>	The layer is configured with a new zone.
Add a sensor	<i>Syntax:</i>	<code>addSensor(zid : String, id : String, loc : String, d : SensorDevice)</code>
	<i>Pre:</i>	No parameter is null and the designated zone does not have a sensor.
	<i>Post:</i>	Adds a sensor to the designated zone. Throws an <code>IllegalStateException</code> if the zone already has a sensor.

Add a valve	<i>Syntax:</i>	<code>addValve(zid: String, id : String, loc : String, rate : int, d : ValveDevice)</code>
	<i>Pre:</i>	No parameter is null and $0 < \text{rate}$ and the designated zone does not have 32 valves.
	<i>Post:</i>	Adds a sensor to the designated zone. Throws an <code>IllegalStateException</code> if the zone already has 32 valves.
Restore program state	<i>Syntax:</i>	<code>restoreState()</code>
	<i>Pre:</i>	Layer configuration is complete.
	<i>Post:</i>	Persistent store is read and the previous state of the program is restored.
Set or get a zone's critical moisture level	<i>Syntax:</i>	<code>setCriticalMoistureLevel(zid : String, level : int)</code> <code>getCriticalMoistureLevel(zid : String) : int</code>
	<i>Pre:</i>	<code>zid</code> is not null and names a registered zone and $0 \leq \text{level} \leq 100$.
	<i>Post:</i>	The designated zone's critical moisture level is set or returned and $0 \leq \text{result} \leq 100$. The change is recorded in persistent store.
Start and end manual irrigation cycles	<i>Syntax:</i>	<code>startManualCycle()</code> <code>endManualCycle()</code>
	<i>Pre:</i>	mode is <code>MANUAL</code>
	<i>Post:</i>	A manual irrigation cycle is started or ended and <code>waterUsed</code> is set to zero when a cycle starts. Throws an <code>IllegalStateException</code> if mode is not <code>MANUAL</code> .
Manually open or close a valve	<i>Syntax:</i>	<code>openValve(id : String)</code> <code>closeValve(id : String)</code>
	<i>Pre:</i>	<code>id</code> is not null and <code>id</code> names a registered valve and mode is <code>MANUAL</code> .
	<i>Post:</i>	A valve is opened or closed. Attempts to open or close failed valves do nothing. Throws an <code>IllegalStateException</code> if mode is not <code>MANUAL</code> .
Obtain irrigation cycle data	<i>Syntax:</i>	<code>getIrrigationReport() : IrrigationReport</code>
	<i>Pre:</i>	None.
	<i>Post:</i>	The <code>IrrigationReport</code> is populated and returned.
Get zone data	<i>Syntax:</i>	<code>getZoneReports() : Collection</code> <code>getZoneReport(id : String) : ZoneReport</code>
	<i>Pre:</i>	<code>id</code> is not null and names a registered zone.
	<i>Post:</i>	If no zone is identified, a collection of <code>ZoneReports</code> , one for each registered zone, is populated and returned. If a zone is identified, a report for that zone is populated and returned.
Get sensor data	<i>Syntax:</i>	<code>getSensorReports() : Collection</code> <code>getSensorReport(id : String) : SensorReport</code>
	<i>Pre:</i>	<code>id</code> is not null and names a registered sensor.
	<i>Post:</i>	If no sensor is identified, a collection of <code>SensorReports</code> , one for each registered sensor, is populated and returned. If a sensor is identified, a report for that sensor is populated and returned.
Get valve data	<i>Syntax:</i>	<code>getValveReports() : Collection</code> <code>getValveReport(id : String) : ValveReport</code>
	<i>Pre:</i>	<code>id</code> is not null and names a registered valve.
	<i>Post:</i>	If no valve is identified, a collection of <code>ValveReports</code> , one for each registered valve, is populated and returned. If a valve is identified, a report for that valve is populated and returned.

Get a failed hardware report	<i>Syntax:</i>	getFailureReport() : FailureReport
	<i>Pre:</i>	None.
	<i>Post:</i>	A new FailureReport is created and populated. It will contain ValveReports and SensorReports only for failed devices.
Mark a device as repaired	<i>Syntax:</i>	repairDevice(id : String)
	<i>Pre:</i>	id is not null and names a registered valve or sensor.
	<i>Post:</i>	The indicated valve or sensor is marked as repaired. The change is recorded in persistent store.

Table B-10-14 Irrigation Layer Interface Specifications**Services Required**

The Irrigation layer uses the following operations from the Device Interface layer: ValveDevice.open(), ValveDevice.close(), SensorDevice.read(), StorageDevice.setData(), StorageDevice.getData(), Clock.getTime(), Clock.setTime(), Clock.getDay(), Clock.setDay(), Clock.addObserver(). The Irrigator is the Clock observer.

Usage Guide

The Irrigation layer should be completely configured using the addX() operations before the restoreState() operation is called. The restoreState() operation uses the Irrigation layer configuration to obtain system state values such as device failure states; if the configuration is not complete, these state values will not be set correctly.

The Irrigator registers itself with the Clock, so a client should not do this.

The various report classes are merely containers for groups of values. They are intended to be used by the User Interface layer to populate screens with data about the state of the system, especially during manual irrigation. They are supposed to provide a wide range of information so they will not have to be changed even if the user interface is changed. Note that because the system is changing during irrigation, the values in the report become stale, so they need be retrieved anew every minute or after every change.

2.5.3 Irrigation Layer Design Rationale

There are many design alternatives that expose the internals of the Irrigation layer by making the Zone, Sensor, and Valve classes part of the layer interface. This makes the Irrigator class much simpler and obviates the need for the many report classes used to return data about the configuration, the state of irrigation, and device states. Such alternatives have two problems: By exposing the internal structure of this layer they make it much harder to change in the future because they do not hide information, and they make it much harder to enforce constraints on irrigation behavior. To illustrate this last point, the alternative previously documented does not allow valves to be manually opened or closed unless the mode is *automatic*. Allowing direct access to valves would make it harder to enforce this constraint. It seems clear that the previous alternative is the best choice because it favors information hiding and control over simplicity.

2.6 User Interface Layer Decomposition

The User Interface layer is responsible for coordinating user interaction with output on a monochrome textual display and input from a 12-key keypad and 8 screen buttons. This layer has a UIController that executes a state machine derived from the user interface dialog map. The controller dispatches input from the keypad, screen buttons, and the clock to the currently active screen.

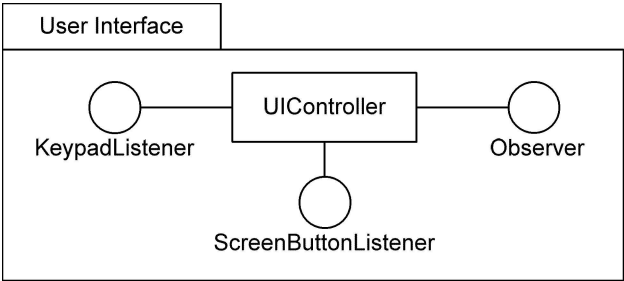


Figure B-10-15 User Interface Layer Structure

2.6.1 User Interface Layer Module Responsibilities

Module	Responsibilities
UIController	Execute a state machine whose states are screens from the dialog map. Dispatch user input and clock notifications to the current screen.

Table B-10-16 User Interface Layer Module Responsibilities

2.6.2 User Interface Layer Interface Specifications

Services Provided

All operations with preconditions on parameters throw `IllegalArgumentException`s if the preconditions are violated.

Create the user interface	<i>Syntax:</i>	<code>create(d : DisplayDevice, w : Irrigator)</code>
	<i>Pre:</i>	<code>d</code> and <code>w</code> are not null.
	<i>Post:</i>	The user interface controller is created and the initial screen in the program is activated.

Table B-10-17 User Interface Layer Interface Specifications

Services Required

The **User Interface** layer uses almost all the operations from the **Irrigation** layer. From the **Device Interface** layer, it uses all the services provided by the **DisplayDevice** and the **Clock**, which it observes. This layer also uses the **KeyPress** enumeration from the **Device Interface** layer.

Usage Guide

The **UIController** must be registered as the listener for the **KeypadDevice** and the **ScreenButtonDevice** to receive notifications from them. It registers itself with the **Clock**.

2.6.3 User Interface Layer Design Rationale

Although the intent is for this layer to implement a state machine realizing the user interface dialog map, the architecture description is so general that any sort of implementation is possible: The **UIController** can be treated as a façade hiding all sorts of alternative implementations. This is perhaps the best reason to prefer this architectural alternative. Other alternatives would provide more detail that would constrain the detailed design alternatives.

2.7 Runtime Components

Figure B-10-18 depicts the runtime configuration of AquaLush.

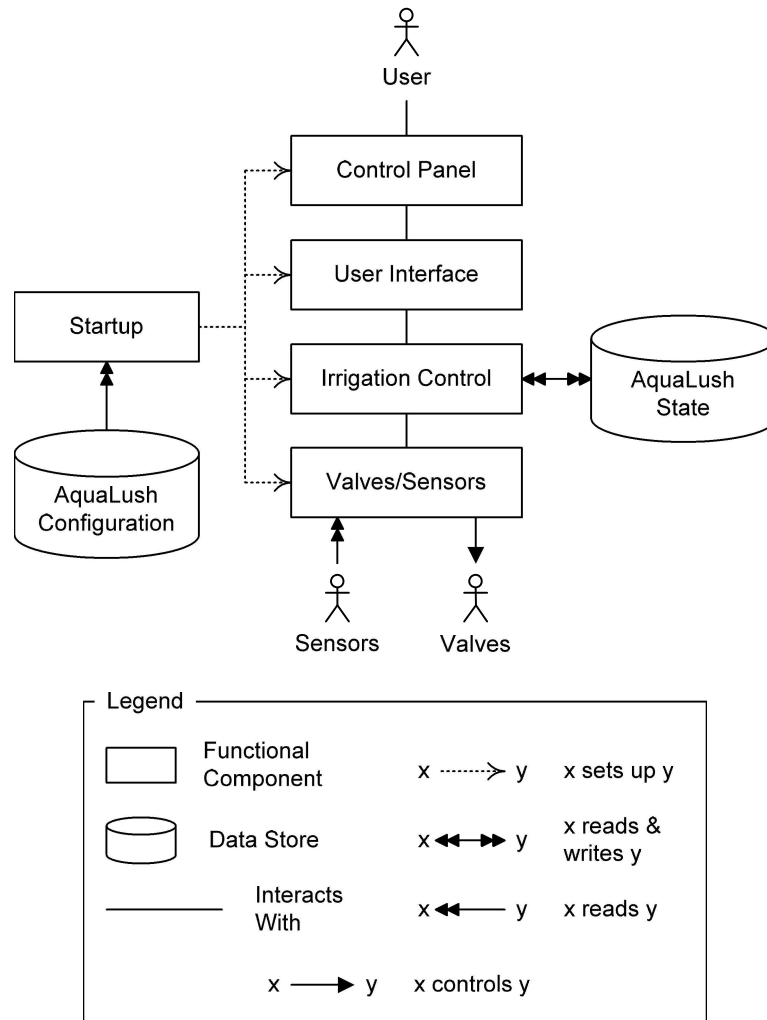


Figure B-10-18 AquaLush Runtime Structure

The **Startup** component executes first and configures the remaining parts of the program using the data it reads from the **AquaLush Configuration** data store. The **User** interacts with the **AquaLush Control Panel**, which interacts with the **User Interface**. The **User Interface** translates the **User**'s desires into interactions with the **Irrigation Control** component to set the mode of the program, set program parameters, and control manual irrigation. The **Irrigation Control** component maintains a copy of its state in the **AquaLush State** data store and uses it to recover its state when it starts up after loss of power. The **Irrigation Control** component interacts with the **Valves/Sensors** component, which reads hardware (or simulated) **Sensors** and controls hardware (or simulated) **Valves**.

3. Mapping Between Models

Table B-10-19 indicates the uses relations between layers and their constituents. The layers in the rows use the classes in the columns.

	User Interface	Irrigation	Device Interface	Simulation
Startup	UIController	Irrigator	DeviceFactory KeypadDevice ScreenButtonDevice	Simulation
User Interface		Irrigator	Clock DisplayDevice	
Irrigation			Clock ValveDevice SensorDevice StorageDevice	
Device Interface				SimTime SimDisplay SimValve SimSensor SimStore

Table B-10-19 Use Relations in AquaLush

The runtime components shown in Figure B-10-18 are comprised of the modules and data stores shown in Table B-10-20.

Runtime Component	Modules
Startup	Startup layer configuration module(s), plus DeviceInterface.DeviceFactory
AquaLush Configuration	Configuration specification (file or applet parameters)
Control Panel	DeviceInterface.DisplayDevice, DeviceInterface.KeypadDevice, and DeviceInterface.ScreenButtonDevice; display, keypad, and screen button hardware or Simulation.SimDisplay and a simulated keypad
User Interface	UserInterface.UIController
Irrigation Control	The objects in the Irrigation layer, plus DeviceInterface.StorageDevice, DeviceInterface.Clock, DeviceInterface.ClockDevice, Simulation.SimStore, or a configuration file, and Simulation.SimTime or real clock hardware
Valves/Sensors	DeviceInterface.ValveDevice, DeviceInterface.SensorDevice
Valves and Sensors	Simulation.SimValve, Simulation.SimSensor, or actual hardware sensors and valves

Table B-10-20 AquaLush Modules and Data Stores

4. Architectural Design Rationale

The layers in this program are rather unusual, but they are arranged as they are to satisfy one of the most important AquaLush quality attributes: configurability. Because the core software, principally the contents of the Irrigation layer, must run unchanged in a simulation or in a fielded product, the program must be configurable so that *all* interactions with its environment can be changed. This includes things that are usually not considered part of the environment, such as time (because simulated time may be faster, slower, or even stop), and things that are usually taken for granted (such as the persistent store, which may not be accessible in an applet).

There is no reasonable alternative except to have a **Device Interface** layer. It uses the **Simulation** layer to provide the entire external environment during a simulation and hardware devices, a file system, and the system clock in a fielded program. In either case, the **Device Interface** layer shields the rest of the program from having to deal with the differences occasioned by these radically different environments.

User interfaces are usually coded using the user interface toolkit provided by a programming environment. However, since the AquaLush simulation uses a GUI and the fielded program uses ATM-like hardware, the user interface must either be coded in several versions or coded once in a device-independent component that uses virtual devices. The latter alternative is clearly preferable. Furthermore, it makes sense to separate the **User Interface** layer from the **Irrigation** layer so that the latter can be unchanged even if the user interface is altered.

Finally, the complexity of the configuration problem suggests the need for a layer responsible for configuring the program at startup. Isolating this task in one place makes it easier to code, test, and modify. The **Startup** layer is responsible for this task.

B.11 AquaLush Detailed Design Document

1. Mid-Level Design Models

The mid-level design models elaborate each architectural layer and are organized in this section by layer.

1.1 Startup Layer Static Structure

The **Startup** layer is responsible for initializing the program as part of a fielded product or as a Web simulation, configuring the program according to the hardware available to it, and restoring the previous program state. The **Startup** layer has the static structure shown in Figure B-11-1.

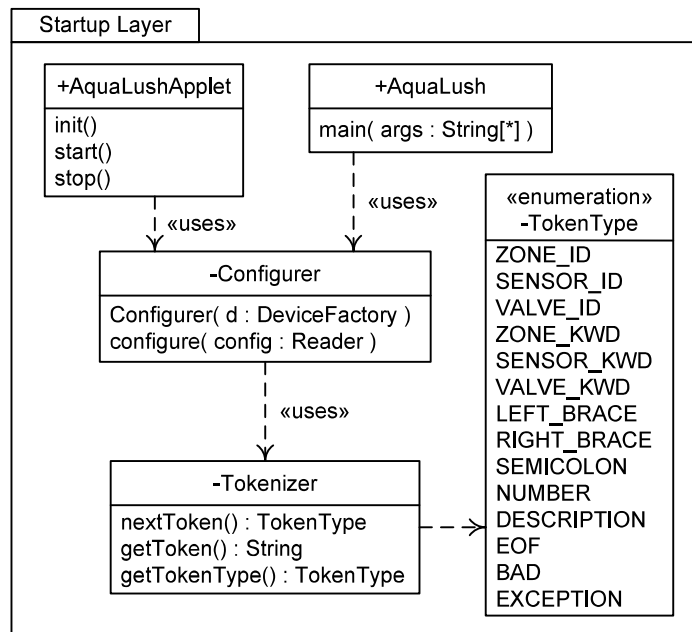


Figure B-11-1 Startup Layer Mid-Level Static Structure